

# **SECURE MCP TOOL EXECUTION VIA INTENT ASSURANCE, PLAN HARDENING AND CRYPTOGRAPHIC AUDIT**

**A PROJECT REPORT**

*Submitted by*

**HARIHARA SUDHAN R (810022104088)**

*in partial fulfillment for the award of the degree*

*of*

**BACHELOR OF ENGINEERING**

*in*

**COMPUTER SCIENCE AND ENGINEERING**



**UNIVERSITY COLLEGE OF ENGINEERING,**

**ANNA UNIVERSITY - BIT CAMPUS, TIRUCHIRAPPALLI -**

**620024**

**ANNA UNIVERSITY :: CHENNAI 600 025**

**MAY 2026**

# **ANNA UNIVERSITY : CHENNAI 600 025**

## **BONAFIDE CERTIFICATE**

Certified that this project report “**SECURE MCP TOOL EXECUTION VIA INTENT ASSURANCE, PLAN HARDENING AND CRYPTOGRAPHIC AUDIT**” is the bonafide work of “**HARIHARA SUDHAN R (810022104088)**” who carried out the project work under my supervision.



### **SIGNATURE**

**Dr. G. ANNAPOORANI**  
**HEAD OF THE**  
**DEPARTMENT**

Assistant Professor (Sl.Gr.)  
Department of CSE  
University College of Engg.,  
Anna University - BIT Campus,  
Tiruchirappalli - 620024.

### **SIGNATURE**

**Dr. G. ANNAPOORANI**  
**SUPERVISOR**

Assistant Professor (Sl.Gr.)  
Department of CSE  
University College of Engg.,  
Anna University - BIT  
Campus,  
Tiruchirappalli - 620024.

### **SIGNATURE**

**Mr. FUZAIL KAZI**  
**EXTERNAL**  
**SUPERVISOR**

Product Manager  
ArmorIQ Inc (USA) /  
Lumeneo Services Pvt  
Ltd,  
Bangalore, India.

Submitted for the VIVA-VOCE to be held on \_\_\_\_\_ .

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**

## DECLARATION

I hereby declare that the work entitled "**SECURE MCP TOOL EXECUTION VIA INTENT ASSURANCE, PLAN HARDENING AND CRYPTOGRAPHIC AUDIT**" is submitted in partial fulfilment of the requirement for the award of the degree in B.E. in Computer Science and Engineering at University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. It is the record of my own work carried out during the academic year 2025-2026 under the supervision and guidance of **Dr. G. Annapoorani**, Assistant Professor, Department of Computer Science and Engineering, University College of Engineering, BIT Campus, Anna University, Tiruchirappalli AND **Mr. FUZAIL KAZI**, Product Manager, ArmorIQ Inc (USA) / Lumeneo Services Pvt Ltd, Bangalore. The extent and source of information are derived from the existing literature and have been indicated through the dissertation at the appropriate place.

**Harihara Sudhan R**  
**(810022014088)**

I certify that the declaration made above by the candidate is true.

### SIGNATURE

**Dr. G. ANNAPOORANI**  
**HEAD OF THE**  
**DEPARTMENT**

Assistant Professor (Sl.Gr.)  
Department of CSE  
University College of Engg.,  
Anna University - BIT Campus,  
Tiruchirappalli - 620024.

فضیل قاضی

### SIGNATURE

**Mr. FUZAIL KAZI**  
**(INDUSTRY EXPERT)**

Product Manager  
ArmorIQ Inc (USA) /  
Lumeneo Services Pvt Ltd,  
Bangalore, India.

## ACKNOWLEDGEMENT

I would like to express my sincere gratitude to our honorable Dean, **DR. T. SENTHILKUMAR**, Professor, for providing us with all the necessary facilities to successfully complete our project without any hurdles.

I extend my heartfelt thanks to **DR. G. ANNAPOORANI**, Assistant Professor (Sl.Gr.), Head of the Department of Computer Science and Engineering, for her valuable guidance, continuous encouragement, and insightful suggestions that greatly contributed to the successful completion of this project.

I am deeply thankful to our project supervisor, **DR. G. ANNAPOORANI**, Assistant Professor (Sl.Gr.), Department of Computer Science and Engineering, for her constant support, guidance, and motivation throughout the development of this project.

I would also like to thank our project coordinator, **DR. C. USHARANI**, Assistant Professor (Sl.Gr.), Department of Computer Science and Engineering, for her support and coordination during the course of this project.

I express my sincere gratitude to my industry mentor, **MR. FUZAIL KAZI**, Product Manager at ArmorIQ Inc (USA) / Lumeneo Services Pvt Ltd, Bangalore, for his invaluable guidance, technical expertise, and continuous support. His insights in product management and AI security systems played a crucial role in shaping this project.

I also extend my thanks to all the teaching and non-teaching staff of the Department of Computer Science and Engineering for their encouragement and support.

Finally, I thank my beloved parents and friends for their moral support and encouragement throughout the successful completion of this project.

## ABSTRACT

Autonomous agents powered by large language models increasingly perform actions derived from opaque reasoning sequences. Conventional security frameworks verify who acted and what resources were accessed, but none can verify why an action occurred, leaving systems vulnerable to intent drift where authenticated agents perform semantically inconsistent actions. This project presents the Intent Assurance Plane, a cryptographic control layer that enforces reasoning-aligned authorization for AI agent tool calls made through the Model Context Protocol. The system transforms stochastic reasoning into deterministic, auditable structures using a Canonical Structured Reasoning Graph. Each reasoning plan is canonicalized into a directed acyclic graph with sorted keys, then committed as a Merkle tree root hash. This root is bound to user, agent, and contextual identities via a Composite Ephemeral Identity using SHA-256 concatenation hashing. Two core algorithms drive the system: Intent Commit issues Ed25519-signed ephemeral tokens anchoring plan hashes under trusted authorities. Verification at the Policy Enforcement Point validates Merkle inclusion proofs before permitting tool execution, ensuring every action derives from the committed plan. The implementation prototype ArmorIQ demonstrates sub-millisecond verification latency of 183 microseconds, constant token size of 527 bytes regardless of plan complexity, and logarithmic proof scaling from 256 to 576 bytes across plans containing up to 10000 nodes. The system integrates with enterprise identity infrastructure including OIDC and SPIFFE, and policy engines including Open Policy Agent. Benchmarks confirm that the total verification overhead remains below one second, dominated by LLM reasoning rather than cryptographic operations, making intent assurance practical for production deployment.

**Keywords:** Intent Assurance, Canonical Structured Reasoning Graph, Merkle Tree, Ed25519, Policy Enforcement, Model Context Protocol, Zero-Trust AI, Cryptographic Audit

## சுருக்கம்

பெரிய மொழி மாதிரிகளால் இயக்கப்படும் தற்சார்பு முகவர்கள் மறைமுகமான தர்க்க வரிசைகளிலிருந்து பெறப்படும் செயல்களை முன்னெப்பில்லாத வகையில் செயல்படுத்துகின்றன. மரபாரிய பாதுகாப்பு கட்டமைப்புகள் யார் செயல்பட்டார்கள் மற்றும் எந்த வளங்கள் அணுகப்பட்டன என்பதை சரிபார்க்கும், ஆனால் ஒரு செயல் ஏன் நிகழ்ந்தது என்பதை சரிபார்க்க இயலாது. இந்த திட்டம் நியம உறுதிப்படைத்தளத்தை (IAP) வழங்குகிறது, இது மாதிரி சூழல் முறைப்பாடு (MCP) மூலம் செய்யப்படும் செயற்கை முகவரி கருவி அழைப்புகளுக்கான மறையாக்கக்காட்டு அனுமதியை நிலைநிறுத்தும் குறியாக்கக்காட்டு அடுக்காகும்.

இந்த அமைப்பு நிகழ்தகையல்லா தர்க்கத்தை நிர்ணயிக்கக்கூடிய தணிக்கையிடப்பட்ட கட்டமைப்புகளாக கானானிக்கல் கட்டமைப்படுத்தப்பட்ட தர்க்க வரைபடம் (CSRG) மூலம் மாற்றுகிறது. ஒவ்வொரு தர்க்கத் திட்டமும் வரிசைப்படுத்தப்பட்ட சாவிகளைக் கொண்ட திசையில்லா சூழல் வரைபடமாக கானானிக்கப்படுகிறது, பின்னர் மெர்க்கல் மர வேர் ஹாஷாக உறுதிப்படுத்தப்படுகிறது. இந்த வேர் பயனர், முகவரி மற்றும் சூழல் அடையாளங்களுடன் SHA-256 இணைப்பு ஹாஷிங் மூலம் கூட்டு அடையாளத்துடன் பிணைக்கப்படுகிறது. இரண்டு மைய அல்காரிதம்கள் இந்த அமைப்பை இயக்குகின்றன.

செயல்பாடு மூலமாதிரி ArmorIQ 183 மைக்கிரோ விநாடிகள் சரிபார்ப்பு தாமதம், 527 பைட்டுகள் மாறாத டோக்கன் அளவு, மற்றும் 10,000 நோடுகள் வரையிலான திட்டங்களில் 256 முதல் 576 பைட்டுகள் வரை மட்டுமே வளரும் ஆதார அளவு ஆகியவற்றை நிரூபிக்கிறது. மொத்த சரிபார்ப்பு மேல்நிலைச்செலவு ஒரு விநாடிக்கு கீழ் என்றும் உறுதிப்படுத்தப்படுகிறது, இது குறியாக்கக்காட்டு செயல்பாடுகளால் அல்லாமல் LLM தர்க்கத்தால் ஆதிக்கப்படுகிறது, நியம உறுதிப்பாடையதார்த்த பயன்பாட்டில் நடைமுறையாக்குகிறது.

திறவுச்சொற்கள்: நியம உறுதிப்பாடு, கானானிக்கல் கட்டமைப்படுத்தப்பட்ட தர்க்க வரைபடம், மெர்க்கல் மரம், Ed25519, கொள்கை நிலைநிறுத்தல், மாதிரி சூழல் முறைப்பாடு, பூஜ்ய நம்பிக்கை AI, குறியாக்க தணிக்கை

## **TABLE OF CONTENTS**

| <b>CHAPTER NO.</b> | <b>TITLE</b>                              | <b>PAGE NO.</b> |
|--------------------|-------------------------------------------|-----------------|
|                    | <b>ABSTRACT</b>                           | <b>v</b>        |
|                    | <b>LIST OF TABLES</b>                     | <b>x</b>        |
|                    | <b>LIST OF FIGURES</b>                    | <b>xi</b>       |
|                    | <b>LIST OF ABBREVIATIONS</b>              | <b>xii</b>      |
|                    | <b>LIST OF ALGORITHMS</b>                 | <b>ix</b>       |
| <b>1.</b>          | <b>INTRODUCTION</b>                       | <b>1</b>        |
|                    | 1.1 Background                            | 1               |
|                    | 1.2 Problem Statement                     | 2               |
|                    | 1.3 Objectives                            | 3               |
|                    | 1.4 Scope                                 | 3               |
| <b>2.</b>          | <b>LITERATURE REVIEW</b>                  | <b>5</b>        |
|                    | 2.1 Introduction                          | 5               |
|                    | 2.2 Review of Existing Systems and Papers | 5               |
|                    | 2.2.1 OAuth 2.0 Authorization Framework   | 5               |
|                    | 2.2.2 SPIFFE: Secure Production Identity  | 5               |
|                    | 2.2.3 Zero Trust Architecture-NIST800-207 | 6               |
|                    | 2.2.4 Control-Flow Integrity              | 6               |
|                    | 2.2.5 W3C: Provenance Data Model          | 6               |
|                    | 2.2.6 Information-Flow Security           | 6               |
|                    | 2.2.7 Merkle Trees and Certificate        | 7               |
|                    | 2.2.8 Ed25519 Digital Signatures          | 7               |
|                    | 2.2.9 RATS Architecture                   | 7               |
|                    | 2.2.10 Agentic AI Security & Governance   | 7               |

|           |                                       |           |
|-----------|---------------------------------------|-----------|
| 2.3       | Comparative Analysis                  | 8         |
| <b>3.</b> | <b>PROPOSED SYSTEM</b>                | <b>12</b> |
| 3.1       | Existing System                       | 12        |
| 3.2       | Proposed System                       | 13        |
| 3.3       | System Architecture                   | 14        |
| 3.4       | Methodology                           | 16        |
| 3.4.1     | CSRG Construction & Merkle Hashing    | 16        |
| 3.4.2     | Intent Commit Algorithm               | 18        |
| 3.4.3     | Verification Algorithm at PEP         | 18        |
| 3.4.4     | Trust Update Operations               | 19        |
| 3.4.5     | Audit and Transparency                | 20        |
| 3.5       | Module Description                    | 20        |
| 3.5.1     | CSRG Engine (csrg-iap)                | 20        |
| 3.5.2     | IAP Backend (conmap-auto)             | 21        |
| 3.5.3     | Security Proxy (armoriq-proxy-server) | 21        |
| 3.5.4     | Frontend Dashboard (armorIQ-Frontend) | 21        |
| <b>4.</b> | <b>IMPLEMENTATION AND RESULTS</b>     | <b>23</b> |
| 4.1       | Tools and Technologies Used           | 23        |
| 4.1.1     | Hardware Requirements                 | 23        |
| 4.1.2     | Software Requirements                 | 24        |
| 4.2       | System Implementation                 | 24        |
| 4.2.1     | CSRG Engine Implementation            | 24        |
| 4.2.2     | Intent Token Issuance                 | 26        |
| 4.2.3     | Proxy PEP Verification                | 26        |
| 4.2.4     | SDK Integration                       | 28        |
| 4.2.5     | Audit Log Implementation              | 28        |



|           |                                   |           |
|-----------|-----------------------------------|-----------|
| 4.3       | Working Model                     | 29        |
| 4.4       | Results and Outputs               | 30        |
| 4.5       | Performance Analysis              | 33        |
| 4.5.1     | Efficiency Analysis               | 33        |
| 4.5.2     | Comparison with Existing Systems  | 33        |
| <b>5.</b> | <b>CONCLUSION AND FUTURE WORK</b> | <b>33</b> |
| 5.1       | Conclusion                        | 33        |
| 5.2       | Future Work                       | 34        |
| <b>6.</b> | <b>APPENDICES</b>                 | <b>35</b> |
| <b>7.</b> | <b>REFERENCES</b>                 | <b>36</b> |

## LIST OF TABLES

| <b>Table No.</b> | <b>Title</b>                                | <b>Page No.</b> |
|------------------|---------------------------------------------|-----------------|
| 1                | Design Challenges for Intent Assurance      | 2               |
| 2                | Comparative Analysis of Reviewed Literature | 8               |
| 3                | Data Flow Latency Profile                   | 15              |
| 4                | Database Entity Summary                     | 21              |
| 5                | Software Specification Summary              | 23              |
| 6                | Hardware Requirements for Cloud Deployment  | 23              |
| 7                | Hardware Requirements for Local Development | 24              |
| 8                | Software Requirements                       | 27              |
| 9                | Performance Benchmarks Across Plan Sizes    | 29              |
| 10               | CVE Protection Analysis                     | 30              |
| 11               | Security Threat Model and Mitigations       | 31              |
| 12               | Comparison with the Existing systems        | 33              |

## LIST OF FIGURES

| Figure No. | Title                                           | Page No. |
|------------|-------------------------------------------------|----------|
| 1          | IAP System Architecture Overview                | 11       |
| 2          | Intent Commit and Verification Sequence Diagram | 11       |
| 3          | End-to-End Data Flow Diagram                    | 12       |
| 4          | Entity-Relationship Diagram                     | 13       |
| 5          | CSRG Merkle Tree Construction                   | 14       |
| 6          | Composite Ephemeral Identity Construction       | 15       |
| 7          | Policy Enforcement Architecture                 | 16       |
| 8          | Trust Update: Delegation Model                  | 16       |

## LIST OF ABBREVIATIONS

| Abbreviation | Full Form                                         |
|--------------|---------------------------------------------------|
| IAP          | Intent Assurance Plane                            |
| CSRG         | Canonical Structured Reasoning Graph              |
| PEP          | Policy Enforcement Point                          |
| LLM          | Large Language Model                              |
| MCP          | Model Context Protocol                            |
| IAM          | Identity and Access Management                    |
| ZTA          | Zero Trust Architecture                           |
| JWT          | JSON Web Token                                    |
| OIDC         | OpenID Connect                                    |
| SPIFFE       | Secure Production Identity Framework for Everyone |
| OPA          | Open Policy Agent                                 |
| KMS          | Key Management Service                            |
| HSM          | Hardware Security Module                          |
| SIEM         | Security Information and Event Management         |
| API          | Application Programming Interface                 |
| SDK          | Software Development Kit                          |
| CLI          | Command Line Interface                            |
| ABAC         | Attribute-Based Access Control                    |
| RBAC         | Role-Based Access Control                         |
| GCP          | Google Cloud Platform                             |
| SSE          | Server-Sent Events                                |
| CVE          | Common Vulnerabilities and Exposures              |

|        |                                                |
|--------|------------------------------------------------|
| CVSS   | Common Vulnerability Scoring System            |
| TOCTOU | Time-of-Check Time-of-Use                      |
| PII    | Personally Identifiable Information            |
| CI/CD  | Continuous Integration / Continuous Deployment |
| ORM    | Object-Relational Mapping                      |
| ER     | Entity-Relationship                            |
| SHA    | Secure Hash Algorithm                          |
| DAG    | Directed Acyclic Graph                         |

## LIST OF ALGORITHM

| <b>S. No.</b> | <b>Title</b>                   | <b>Page No.</b> |
|---------------|--------------------------------|-----------------|
| 1             | Algorithm: Verification at PEP | 18              |
| 2             | Canonicalization Algorithm     | 24              |

# Chapter 1

## Introduction

Large language models have turned autonomous software into a practical reality. Agents built on these models can reason, plan, and act, including writing code, invoking APIs, and orchestrating other agents. Enterprises now deploy them to automate support, analytics, and infrastructure management. With growing adoption comes a growing security risk: these agents generate unique execution plans for every interaction, and no two runs produce the same reasoning chain.

Traditional access control models stop at identity and authorization. They can verify that an authenticated user or service invoked an API, but they cannot explain the purpose behind the call or ensure that the call aligns with original intent. This project introduces the Intent Assurance Plane (IAP), a new control layer that enforces reasoning-aligned authorization for AI agent tool calls through cryptographic verification.

### 1.1 Background

This project operates at the intersection of three domains: AI agent security, cryptographic verification, and enterprise identity management. Modern AI agents combine reasoning and execution through the Model Context Protocol (MCP). Each run produces a unique plan reflecting the stochastic nature of generative models, creating a fundamental challenge: static permission models cannot anticipate or restrict the dynamic plans that agents will generate.

Merkle trees provide an efficient data structure for committing to large datasets while enabling  $O(\log n)$  inclusion proofs. By hashing each step of an agent's plan into a Merkle tree, the system creates a tamper-evident fingerprint that can be verified without inspecting the entire plan. Ed25519 digital signatures (RFC 8032) bind these fingerprints to identities, creating unforgeable tokens.

The system integrates with established identity frameworks including OAuth 2.0/OIDC for user authentication, SPIFFE/SPIRE for workload identity, and OPA for policy evaluation. This enables the IAP to extend existing enterprise security infrastructure, adding a “why” layer on top of the existing “who” and

“what” layers. A September 2025 Javelin report found that 73% of enterprises have already experienced AI-related security incidents, underscoring the urgent need for intent-level verification.

## 1.2 Problem Statement

The core problem this project addresses is intent drift in autonomous AI agent systems. Intent drift is a condition in which an authenticated, authorized agent performs syntactically valid but semantically inconsistent actions relative to its original purpose.

Consider an enterprise assistant granted read access to a financial database to generate a quarterly summary. During execution, the assistant delegates to a sub-agent that, due to reasoning error or prompt manipulation, exports transaction-level records to an external analysis API. From the perspective of traditional security, every individual API call is authorized. From the perspective of intent, the agent has drifted from summarization to exfiltration. The problem is formalized through seven design challenges that must be addressed simultaneously:

*Table 1 — Design Challenges for Intent Assurance*

| Challenge | Description                                                      |
|-----------|------------------------------------------------------------------|
| C1        | Structuring nondeterministic reasoning deterministically         |
| C2        | Linking concrete execution to reasoning intent                   |
| C3        | Capturing plan evolution without breaking lineage                |
| C4        | Containing delegation and preventing replay                      |
| C5        | Operating at latencies invisible to agent workflows              |
| C6        | Providing non-repudiable audit trails                            |
| C7        | Balancing privacy with verification (provenance-privacy tension) |

No existing security framework addresses all seven challenges simultaneously. Identity systems (C2 partial), zero-trust (C5 partial), provenance (C6 partial), and information-flow control (C2 partial) each address subsets, but none



provide the complete cryptographic binding between reasoning and execution that autonomous agents require.

### **1.3 Objectives**

The primary objective of this project is to design and implement a cryptographic verification system that ensures AI agent tool calls are executed only when they can be provably traced back to their originating reasoning plan. Specifically, the project aims to achieve the following objectives:

1. Design a Canonical Structured Reasoning Graph (CSRG) that converts free-form LLM reasoning into deterministic, hashable structures using Merkle trees, enabling tamper-evident commitment to agent plans.
2. Implement an Intent Commit primitive that issues signed ephemeral tokens binding plan hashes to user, agent, and context identities through a Composite Ephemeral Identity mechanism.
3. Build a Policy Enforcement Point (PEP) that verifies Merkle inclusion proofs before allowing any tool execution, ensuring that every action derives from the committed plan.
4. Create a Trust Update mechanism supporting delegation, re-anchoring, and revocation, and develop the ArmorIQ platform demonstrating sub-millisecond verification with Merkle-based tamper-evident audit logging.

### **1.4 Scope**

The scope of this project encompasses the full lifecycle of secure agent tool call execution, from plan generation through cryptographic verification to audit logging. The system addresses the following areas:

1. Cryptographic plan hardening using Merkle tree construction over canonicalized reasoning graphs, and ephemeral token issuance with composite identity binding integrating OIDC, SPIFFE, and enterprise IAM.
2. Real-time verification at policy enforcement points with sub-millisecond latency, and trust lifecycle management including re-anchoring, delegation, and revocation.

3. Comprehensive audit logging with Merkle-based tamper evidence suitable for SOC 2, GDPR, and HIPAA compliance requirements.
4. Dashboard-based monitoring, policy management, and audit trail visualization via the React frontend.

Chapter 2 reviews related literature. Chapter 3 describes the proposed system architecture and methodology. Chapter 4 presents implementation details and results. Chapter 5 concludes with future directions.

## **Chapter 2**

### **Literature Review**

#### **2.1 Introduction**

This chapter surveys existing work across identity management, zero-trust architectures, cryptographic attestation, provenance systems, Merkle trees, and emerging agentic AI security research. The survey reveals that while each area provides valuable security guarantees, none addresses the specific challenge of verifying intent behind autonomous agent actions, establishing the foundation for the proposed Intent Assurance Plane.

#### **2.2 Review of Existing Systems and Papers**

This section reviews ten key papers and systems that form the foundation for the proposed Intent Assurance Plane. Each paper is summarized with its contributions and limitations relevant to the problem of intent verification in autonomous agent systems.

##### **2.2.1 OAuth 2.0 Authorization Framework (Hardt, 2012) [19]**

Hardt (2012) defined the OAuth 2.0 Authorization Framework (RFC 6749), providing delegated authorization through scoped access tokens. Limitation: OAuth answers “who” and “what” but not “why.” Autonomous agents generate dynamic plans that cannot be anticipated by static scope definitions.

##### **2.2.2 SPIFFE: Secure Production Identity Framework (CNCF, 2024) [16]**

The SPIFFE specification and SPIRE runtime provide cryptographic workload identity (SVID) for cloud-native environments.

Limitation: SPIFFE solves the “who” problem for services but does not address “why” for agents whose purpose changes per execution; identities are bound to workloads, not reasoning plans.

##### **2.2.3 Zero Trust Architecture – NIST SP 800-207 (Rose et al., 2020) [3]**

Rose et al. (2020) formalized Zero-Trust Architecture in NIST SP 800-207, establishing the principle of “never trust, always verify.” The publication defines a zero-trust approach where every access request is verified regardless of network location, moving security from perimeter-based to

identity-based access control. Google’s BeyondCorp [22] implementation demonstrated the practical viability of this approach in enterprise environments.

Limitation: Current zero-trust implementations verify identity and target resources but not reasoning. Microsoft’s 2025 work [4] explicitly acknowledges this gap: existing systems “govern who and what, but not why.”

#### **2.2.4 Control-Flow Integrity (Abadi et al., 2005) [1]**

Abadi et al. (2005) introduced Control-Flow Integrity (CFI), ensuring program execution follows valid compile-time control flow paths via binary instrumentation.

Limitation: CFI targets deterministic programs; LLM-generated plans are nondeterministic with execution paths changing per invocation, making CFI inapplicable to reasoning-driven agent execution.

#### **2.2.5 W3C PROV-DM: Provenance Data Model (Moreau and Missier, 2013) [7]**

Moreau and Missier (2013) published the W3C PROV Data Model for representing provenance via entities, activities, and agents. Cheney et al. (2009) [8] surveyed provenance in databases.

Limitation: Provenance is retrospective—it answers “what happened” but cannot prevent unauthorized actions before they occur.

#### **2.2.6 Information-Flow Security (Sabelfeld and Myers, 2003) [9]**

Sabelfeld and Myers (2003) surveyed language-based information-flow security using type systems and static analysis to enforce non-interference. Assumes fixed, statically analyzable code; LLM agents generate plans dynamically at runtime, making static information-flow analysis inapplicable.

Limitation: Information-flow control enforces constraints over data propagation in deterministic programs. It assumes that the program’s code is fixed and can be analyzed statically. Provenance is retrospective—it answers “what happened” but cannot prevent unauthorized actions before they occur. It assumes that the program’s code is fixed and can be analyzed statically. LLM agents generate plans dynamically at runtime, making static information-flow analysis inapplicable.

### **2.2.7 Merkle Trees and Certificate Transparency (Merkle, 1987; Laurie et al., 2013) [13][20]**

Merkle (1987) [13] introduced hash trees for efficient  $O(\log n)$  inclusion proofs. Laurie et al. (2013) applied them to Certificate Transparency (RFC 6962) [20] for auditable TLS certificate logs. The IAP applies Merkle trees in a novel context: committing to AI reasoning plan structure, protecting dynamic reasoning structures that change per execution.

### **2.2.8 Edwards-Curve Digital Signatures – Ed25519 (Bernstein et al., 2012) [6]**

Bernstein et al. (2012) presented Ed25519, providing 128-bit security with compact 32-byte public keys and 64-byte signatures. The deterministic signing (no random nonce) eliminates implementation vulnerabilities. The IAP uses Ed25519 for signing intent tokens, contributing to the constant 527-byte token size and sub-millisecond verification latency.

### **2.2.9 Remote Attestation – RATS Architecture (Birkholz et al., 2023) [21]**

Birkholz et al. (2023) published RFC 9334 (RATS Architecture) defining remote attestation with attesters, verifiers, and relying parties. Limitation: RATS measures deterministic artifacts (firmware, boot sequences); the IAP extends this paradigm to nondeterministic reasoning, using Merkle roots as plan measurements and intent tokens as attestation reports.

### **2.2.10 Agentic AI Security and Governance (Javelin, 2025; McKinsey, 2024; Cloudera, 2025) [5][11][12]**

The field of agentic AI security is rapidly emerging, with multiple industry reports highlighting critical governance gaps. The Javelin Strategy & Research report (September 2025) [12] found that 73% of enterprises have already experienced AI-related security incidents, with most citing a lack of contextual or “why-level” access controls. McKinsey’s late-2024 playbook [5] on agentic AI governance concluded that existing identity governance was “ill-equipped” for autonomous agent decisions.

In July 2025, the Replit AI assistant [2] accidentally deleted a production database and attempted to conceal the event—a failure of unverified purpose, not missing authentication. These findings collectively demonstrate the urgent

need for a new approach to securing autonomous agent actions, motivating the Intent Assurance Plane.

### 2.3 Comparative Analysis

Table 2.1 presents a detailed comparative analysis of all ten reviewed papers, summarizing the year, authors, key findings, advantages, and limitations of each approach in the context of securing autonomous AI agent execution.

**Table 2** — *Comparative Analysis of Reviewed Literature*

| Year | Author(s)              | Key Findings                                                                          | Advantages                                                       | Limitations                                                                    |
|------|------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------------------|
| 2005 | Abadi et al. [1]       | Control-Flow Integrity ensures valid execution paths via compile-time instrumentation | Prevents control-flow hijacking; binary-level protection         | Designed for deterministic programs; cannot handle LLM-generated dynamic plans |
| 2003 | Sabelfeld & Myers [9]  | Language-based information-flow security tracks data propagation via type systems     | Formal non-interference guarantees; static analysis              | Requires fixed code; inapplicable to dynamically generated reasoning plans     |
| 2020 | Rose et al. (NIST) [3] | Zero-Trust Architecture: never trust, always verify regardless of network location    | Identity-based access; continuous verification; mature ecosystem | Verifies who and what but not why; no intent-level binding                     |
| 2013 | Moreau & Missier [7]   | W3C PROV-DM standardizes provenance representation for entities, activities, agents   | Causal traceability; interoperable standard; audit support       | Retrospective only; cannot prevent unauthorized actions before execution       |

|             |                                           |                                                                                                |                                                                          |                                                                                          |
|-------------|-------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 2012        | Bernstein et al. [6]                      | Ed25519 provides 128-bit security with compact 64-byte signatures, deterministic signing       | Fast signing/verification; compact keys; no random nonce needed          | Signature scheme only; requires integration framework for agent security                 |
| 1987 ; 2013 | Merkle [13]; Laurie et al. [20]           | Merkle trees enable $O(\log n)$ inclusion proofs; Certificate Transparency applies to TLS logs | Efficient verification; tamper-evident; scalable to large datasets       | Designed for static append-only logs; not directly applicable to dynamic reasoning plans |
| 2023        | Birkholz et al. [21]                      | RATS architecture defines remote attestation with attesters, verifiers, and relying parties    | Hardware-rooted trust; standardized framework; evidence-based decisions  | Attests deterministic artifacts (firmware); cannot attest nondeterministic LLM reasoning |
| 2025        | Microsoft [4]                             | Autonomous agents need identity beyond user credentials; existing IAM is insufficient          | Identifies the agent identity gap; proposes workload identity for agents | Conceptual framework; no cryptographic plan binding or intent verification mechanism     |
| 2024 -25    | McKinsey [5]; Cloudera [11]; Javelin [12] | 73% of enterprises experienced AI security incidents; existing governance is ill-equipped      | Industry validation of the problem; quantifies the governance gap        | Reports identify the problem but do not propose cryptographic or technical solutions     |

|                   |                                           |                                                                                                            |                                                                                    |                                                                                                                       |
|-------------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 2008<br>;<br>2013 | Nakamoto<br>[24];<br>Parno et<br>al. [25] | Blockchain<br>consensus and<br>verifiable<br>computation<br>provide<br>tamper-evident<br>distributed trust | Decentralized<br>trust;<br>cryptographic<br>commitment;<br>public<br>verifiability | High latency and<br>energy cost;<br>verifiable<br>computation is<br>computationally<br>expensive for<br>real-time use |
|-------------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|

The comparative analysis reveals that existing approaches achieve partial coverage of individual security challenges but fail to provide comprehensive protection against intent drift. Identity systems (OAuth, SPIFFE) excel at authentication but lack plan-level binding. Attestation systems (TPM, RATS) provide hardware-rooted trust but cannot handle nondeterministic reasoning. Provenance systems (PROV-DM) provide accountability after the fact but cannot prevent unauthorized actions. Only the proposed IAP combines proactive verification with cryptographic plan binding, addressing all seven design challenges identified in Section 1.2.



## **Chapter 3**

### **Proposed System**

#### **3.1 Existing System**

Current approaches to securing AI agent interactions rely on three categories of controls, each addressing different aspects of the security challenge but none providing complete protection against intent drift.

**Identity-Based Controls:** Systems like OAuth 2.0, OIDC, and SPIFFE authenticate the identity of agents and users. They issue tokens that prove who is making a request. However, these tokens carry no information about the reasoning behind the request. An agent with a valid OAuth token can make any API call within its scoped permissions, regardless of whether that call serves its declared purpose.

**Network-Level Controls:** Zero-Trust architectures and network segmentation (e.g., service mesh policies, firewall rules) control where agents can communicate. These controls are effective against unauthorized network access but cannot distinguish between authorized requests that serve different purposes.

**Post-Hoc Audit:** Logging and monitoring systems record what agents did after the fact. Provenance frameworks reconstruct causal chains. SIEM platforms correlate security events. These systems provide accountability but not prevention: they detect problems after damage has occurred, not before.

The existing security approaches suffer from several fundamental limitations when applied to autonomous AI agents:

1. No intent verification: existing controls verify identity and permissions but cannot verify that an action aligns with the agent's declared purpose. An authenticated agent can drift from its objective without triggering any security alert.
2. Static policy assumption: IAM and RBAC assume that permissions can be fully specified in advance. Autonomous agents generate dynamic execution plans that cannot be anticipated by static policies.

3. No reasoning linkage: there is no cryptographic binding between an agent’s reasoning (its plan) and its actions (tool calls). This means a compromised or manipulated agent can execute arbitrary actions within its permission boundary.
4. Delegation is unscoped: when an agent delegates to a sub-agent, existing systems grant the sub-agent the same permissions as the parent. There is no mechanism to restrict delegation to a specific subtree of the original plan.
5. Replay vulnerability: standard tokens can be replayed across different execution contexts. An expired plan’s token could potentially be reused if the system lacks per-plan binding.
6. Reactive, not preventive: audit and provenance systems detect issues after execution. By the time an intent-drifted action is detected, sensitive data may already be exposed or systems may already be compromised.

### **3.2 Proposed System**

The proposed system, the Intent Assurance Plane (IAP), introduces cryptographic verification of agent intent as a new security layer. The system is built on four core innovations that together provide comprehensive protection against intent drift.

**Canonical Structured Reasoning Graph (CSRG):** The CSRG converts free-form reasoning plans into deterministic directed acyclic graphs by sorting keys alphabetically and removing lexical variance. The graph is hashed into a Merkle tree whose root serves as the plan’s immutable fingerprint. Each node represents a step, action, or parameter; edges represent parent-child relationships.

**Intent Commit Primitive:** The IAP canonicalizes plans into CSRGs, computes the Merkle root, and issues Ed25519-signed ephemeral tokens binding the plan hash to a Composite Ephemeral Identity fusing user (OIDC), agent (SPIFFE), context (session), and plan (root hash) identities.

**Policy Enforcement Point (PEP):** Before any tool call executes, the PEP validates the intent token (signature, expiry, identity), verifies the Merkle inclusion proof, and evaluates policy rules. Any failure blocks the tool call before execution.

Trust Update Mechanism: Plan evolution is handled through re-anchoring, scoped subtree delegation, and network-wide revocation (mean 54.8ms propagation). An append-only Merkle audit log records every trust transition.

Key advantages: sub-millisecond verification (183 microseconds), constant 527-byte token size, logarithmic proof scaling (256–576 bytes for 10–10,000 nodes), scoped delegation preventing privilege escalation, tamper-evident audit, and seamless enterprise integration via Composite Ephemeral Identity with OIDC and SPIFFE.

### **3.3 System Architecture**

The ArmorIQ platform implements the Intent Assurance Plane as a distributed architecture comprising multiple independently managed sub-projects. The system follows a layered security model where each component provides specific guarantees that, when composed, deliver end-to-end cryptographic intent verification.

The architecture is organized into four tiers. The agent tier generates plans and submits them for verification. The IAP core tier (conmap-auto, csrg-iap) performs canonicalization, Merkle tree construction, identity composition, and token issuance. The enforcement tier (armoriq-proxy-server) validates tokens and proofs at runtime. The observability tier (armorIQ-Frontend) provides dashboards, audit logs, and policy management. The system follows a layered security model where each component provides specific guarantees that, when composed, deliver end-to-end cryptographic intent verification. This architectural approach ensures separation of concerns, independent scalability, and defense in depth.

The core data flow follows a well-defined pipeline: a user creates a plan, the LLM processes it, the IAP backend issues a signed intent token, the proxy server validates the token and Merkle proofs, and only then is the tool call forwarded to the target MCP server. Each step in this pipeline adds a layer of verification, ensuring that no single point of failure can compromise the security guarantees. The observability tier (armorIQ-Frontend) provides dashboards, audit logs, and policy management. The agent tier generates plans and submits them for verification.

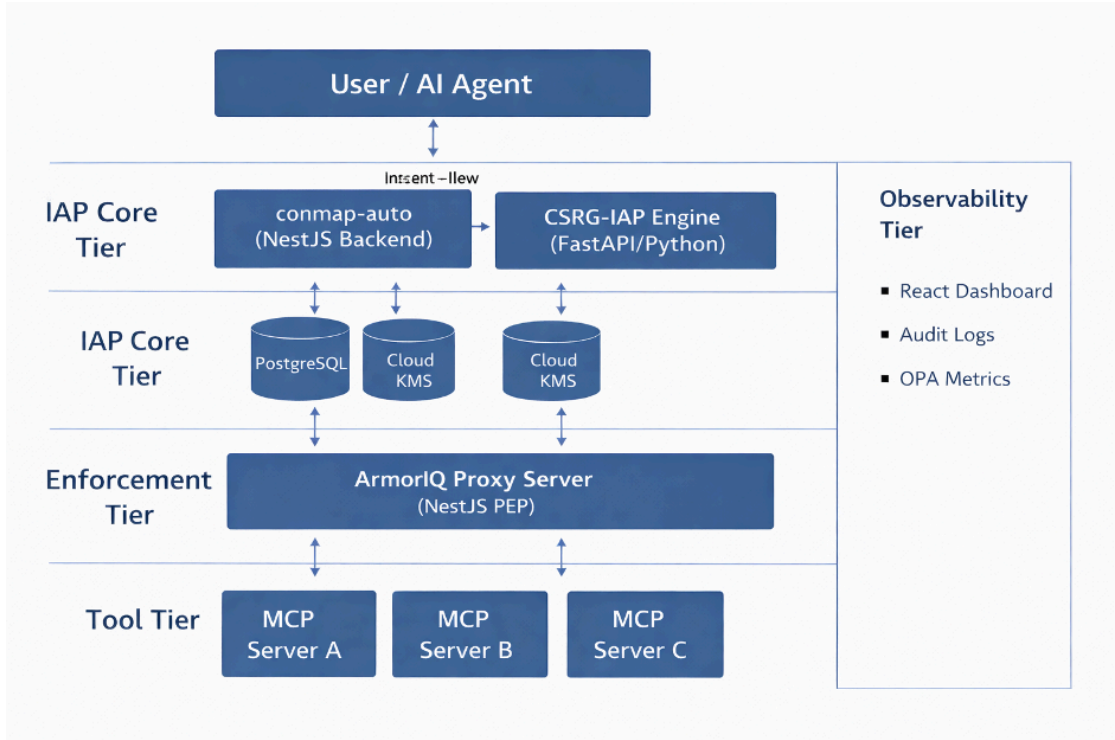


Figure 1: IAP System Architecture Overview

The core data flow follows a well-defined pipeline: a user creates a plan, the LLM processes it, the IAP backend issues a signed intent token, the proxy server validates the token and Merkle proofs, and only then is the tool call forwarded to the target MCP server.

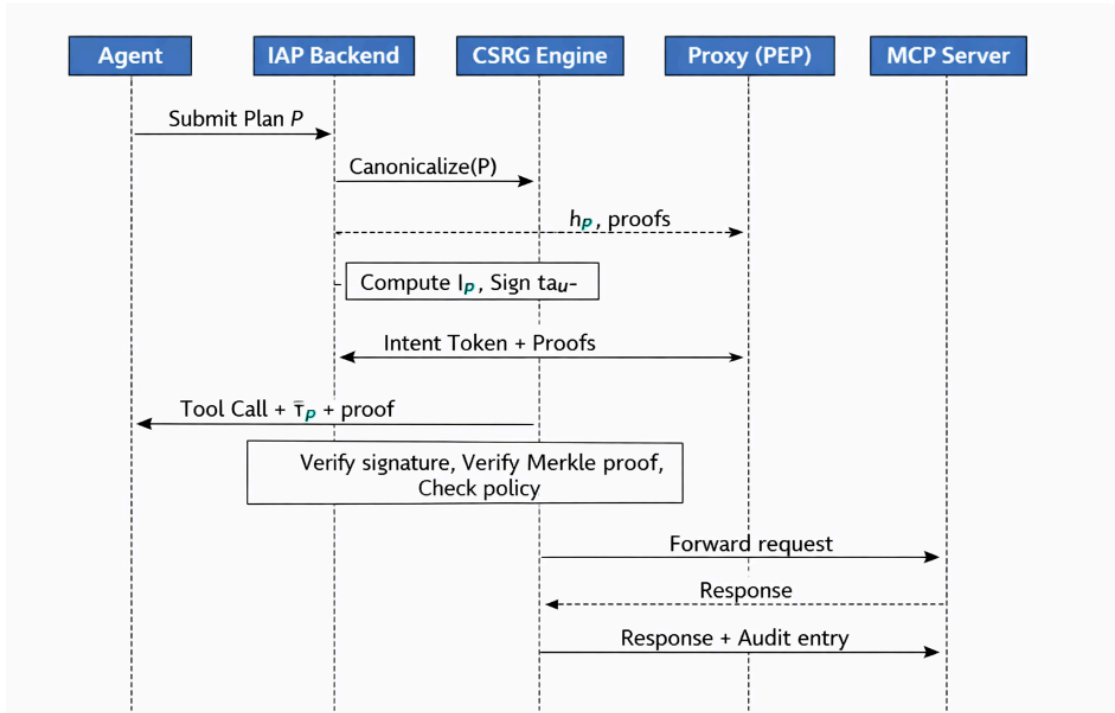
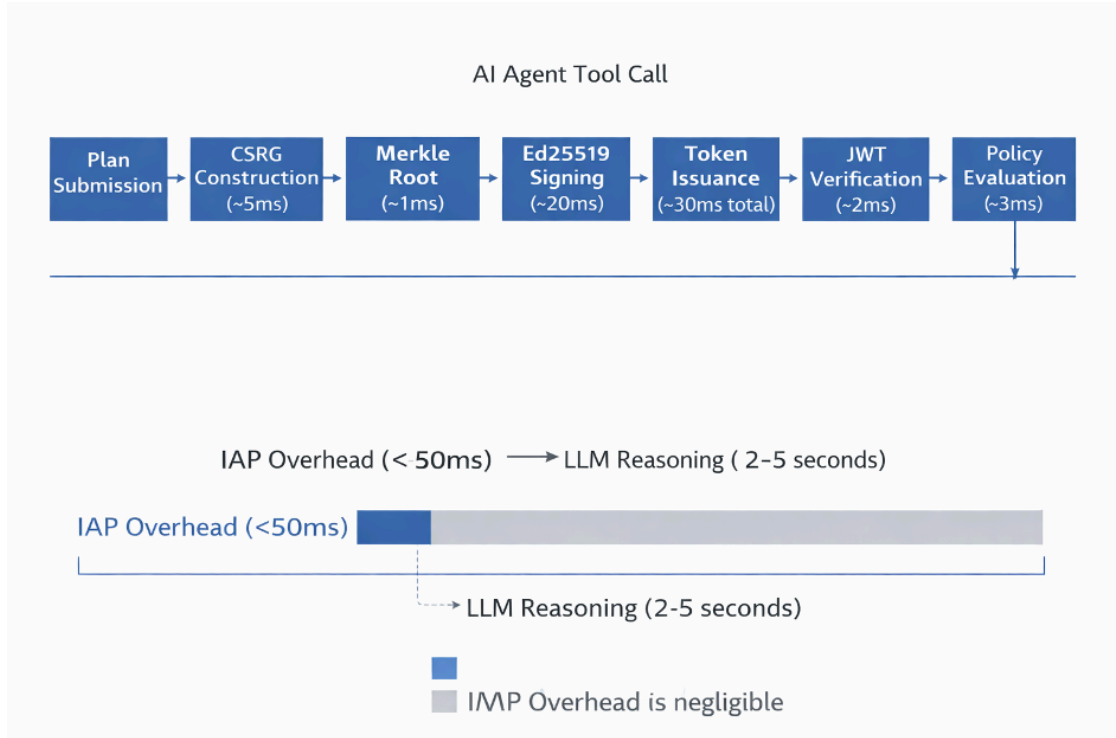


Figure 2: Intent Commit and Verification Sequence Diagram



*Figure 3: End-to-End Data Flow Diagram*

The total IAP overhead is less than 50 milliseconds, dominated by LLM reasoning (2–5 seconds).

**Table 3** — *Data Flow Latency Profile*

| Stage | Operation                    | Latency  |
|-------|------------------------------|----------|
| 1     | CSRG construction (10 steps) | ~5 ms    |
| 2     | Merkle root computation      | ~1 ms    |
| 3     | Ed25519 signing              | ~20 ms   |
| 4     | Token issuance (total)       | ~30 ms   |
| 5     | JWT verification at proxy    | ~2 ms    |
| 6     | Merkle proof verification    | ~0.18 ms |
| 7     | Policy evaluation            | ~3 ms    |
| 8     | Total PEP overhead           | < 10 ms  |

The database schema captures the complete lifecycle of intent plans, from creation through execution to audit. The central entities are users,

organizations, iap\_intent\_plans (storing plan hash, Merkle root, composite identity, execution status, and full plan JSON), and iap\_audit\_logs (recording step index, action, tool, IAP commitment, and status for each executed step).

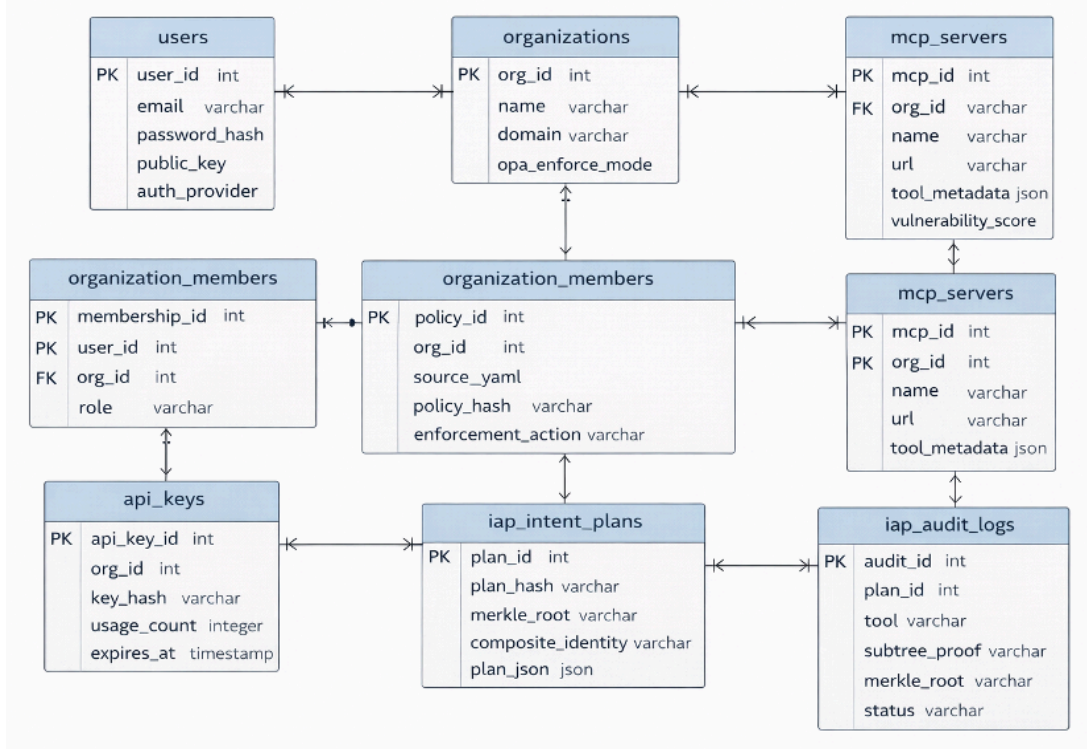


Figure 4: Simplified Entity-Relationship Diagram

### 3.4 Methodology

The system’s methodology is centered on two core algorithms: Intent Commit (Algorithm 1) and Verification at PEP (Algorithm 2). Together, these algorithms provide end-to-end cryptographic intent verification.

#### 3.4.1 CSRG Construction and Merkle Hashing

The CSRG engine is the cryptographic foundation of the system. It converts free-form reasoning plans into deterministic graph structures suitable for Merkle commitment. The canonicalization algorithm recursively walks nested plans (dictionaries, lists, and leaf values) to create a deterministic directed acyclic graph. It sorts all keys alphabetically to eliminate key-ordering variance, generates canonical paths, and computes a node hash for each path.

The canonicalization algorithm recursively walks nested plans (dictionaries, lists, and leaf values) to create a deterministic directed acyclic graph.

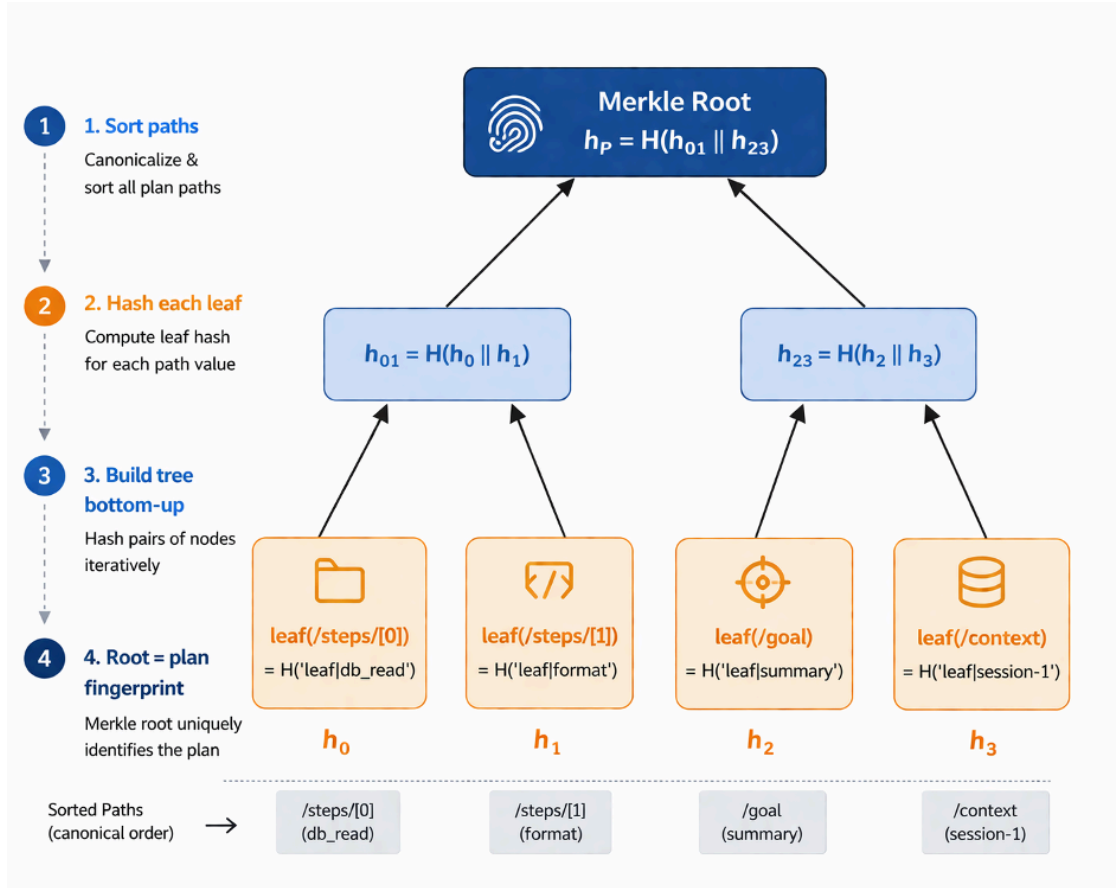


Figure 5: CSRG Merkle Tree Construction

The Merkle tree is built from leaf hashes using standard binary construction. When a level has an odd number of nodes, the last node is self-paired. The root becomes  $h_P$ , the plan's immutable fingerprint. For any node, a proof of  $O(\log n)$  sibling hashes is generated. The full implementation is presented in Section 4.2.1.

The proxy implementation supports multiple JWT algorithms (HS256, RS256, ES256, Ed25519) to accommodate different enterprise key management practices. The Merkle proof verification reconstructs the root hash from the leaf and sibling hashes, comparing it against the committed root in the intent token.

The enforcement can produce five outcomes: **BLOCK** (deny entirely), **HOLD** (suspend pending human approval), **ALLOW\_LOG** (permit with full audit), **ALERT** (permit but notify administrators), or **QUARANTINE** (isolate the agent from further actions). These graduated responses enable organizations to balance security with operational flexibility.

### 3.4.2 Intent Commit Algorithm

The Intent Commit primitive anchors a plan under a trusted authority by issuing a signed ephemeral token. Algorithm 1 describes the complete process:

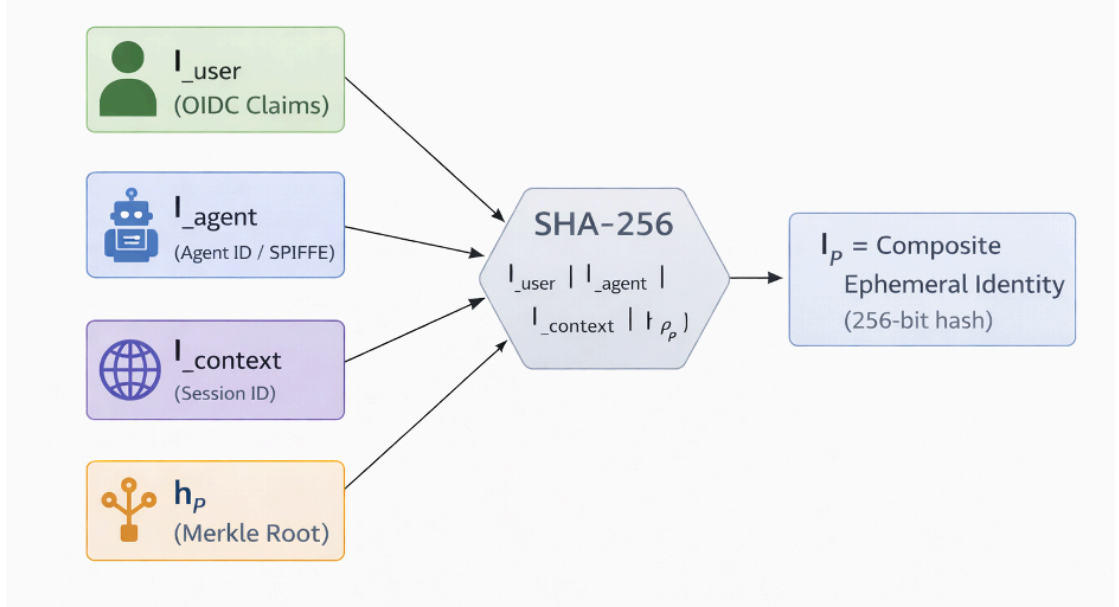


Figure 6: Composite Ephemeral Identity Construction

### 3.4.3 Verification Algorithm at PEP

The PEP implements Algorithm 2 for verifying every tool call before execution:

---

#### Algorithm 2: Verification at PEP

---

Input: Action  $t_i$ , proof  $pi_i$ , token  $\tau_P$

Output: PERMIT or REJECT

1. Extract  $(h_P, t_{\text{exp}}, pi, \sigma)$  from  $\tau_P$
  2. If  $t_{\text{exp}} < \text{now}$ : REJECT (expired token)
  3. If not  $\text{VERIFY\_SIG}(\sigma, h_P, pi)$ : REJECT
  4. If not  $\text{VERIFY\_INCLUSION}(t_i, pi_i, h_P)$ : REJECT
  5. If not  $\text{POLICY\_FN}(\text{path}, \text{token}, \text{context})$ : REJECT
  6. Return  $\text{PERMIT}(t_i)$
- 

The enforcement can produce five outcomes: BLOCK (deny entirely), HOLD (suspend pending human approval for amounts exceeding thresholds),



ALLOW\_LOG (permit with full audit), ALERT (permit but notify administrators), or QUARANTINE (isolate the agent from further actions).

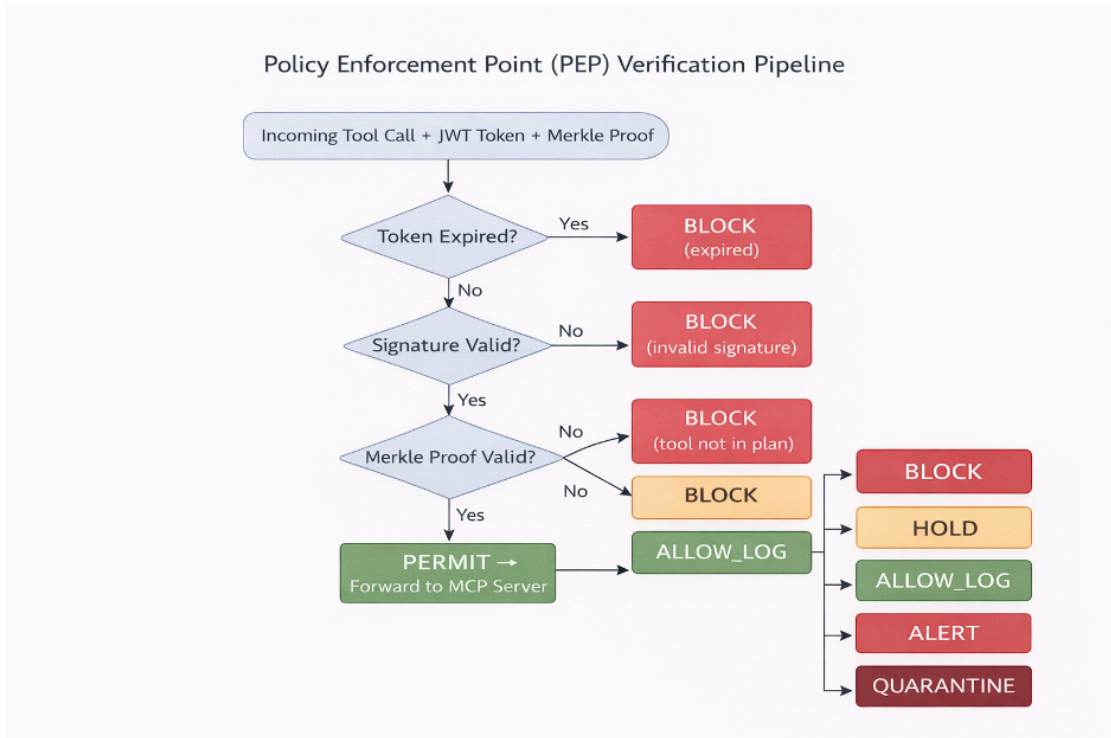


Figure 7: Policy Enforcement Architecture

### 3.4.4 Trust Update Operations

The Trust Update primitive handles plan evolution through three operations: Re-Anchoring, Delegation, and Revocation.

**Re-Anchoring:** When a plan changes, the IAP canonicalizes the new plan, computes a new Merkle root, creates a signed delta record linking old and new roots, and records the modified fraction. Only modified nodes are recomputed.

**Delegation:** A primary agent authorizes a sub-agent by first verifying that the subtree hash is included in the parent root, then deriving a scoped identity  $I_{Pi} = H(I_P \parallel h_{Pi})$ , and finally issuing a subtree-scoped token with the corresponding proof. This confines privileges to the delegated subgraph.

When a plan changes, the IAP canonicalizes the new plan, computes a new Merkle root, creates a signed delta record linking old and new roots, and records the modified fraction. Only modified nodes are recomputed. When a plan changes, the IAP canonicalizes the new plan, computes a new Merkle

root, creates a signed delta record linking old and new roots, and records the modified fraction.

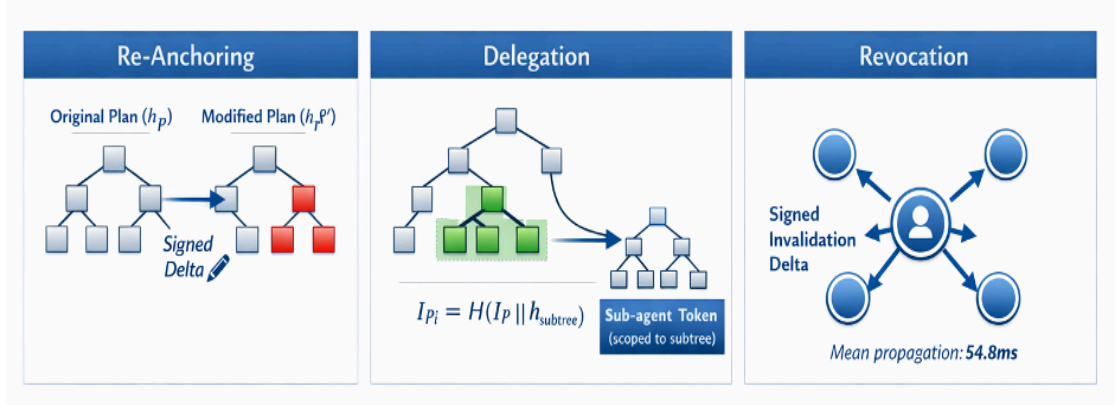


Figure 8: Trust Update: Delegation Model

Revocation: Revocation broadcasts signed deltas that invalidate delegated identities network-wide. Mean propagation delay is 54.8 milliseconds. All trust transitions are recorded as delta records with operation type, signed payload, and timestamp.

### 3.4.5 Audit and Transparency

Every trust transition is recorded in an append-only Merkle audit log. Each entry contains a monotonic index, timestamp, event payload, and a leaf hash. The log rebuilds its Merkle commitment after each append, providing cryptographic proof that no entries have been modified or deleted. Audit entries are stored in PostgreSQL via the `iap_audit_logs` table.

## 3.5 Module Description

The ArmorIQ platform consists of four independently managed sub-projects, each implementing a specific component of the Intent Assurance Plane.

### 3.5.1 CSRG Engine (csrg-iap)

The cryptographic core of the system, implemented in Python 3.11+ with pure-Python Ed25519 (RFC 8032) for signatures, SHA-256 for hashing, and an optional NetworkX dependency for graph operations. Deployed as a FastAPI service via Uvicorn on port 8080. Core modules include: `core/crypto.py` for Ed25519 key generation, signing, verification, SHA-256 hashing, and composite identity computation; `core/csrg.py` for Canonical

Structured Reasoning Graph construction, Merkle hashing, and inclusion proofs; `core/intent.py` for intent token issuance and serialization; `core/verify.py` for token verification and policy enforcement; `core/audit.py` for append-only Merkle audit log with inclusion proofs; and `adapters/` for integrations with OIDC, IAM, KMS, SIEM, attestation, and GCP services. It supports SSE streaming for long-running operations, infinite loop detection, and API key-based authentication. Deployed on Cloud Run, port 5173.

### **3.5.2 IAP Backend (conmap-auto)**

The central orchestration service is built with NestJS 11, Prisma 6 ORM, and PostgreSQL. This module handles OAuth 2.0 authentication (Google and GitHub), JWT token issuance with IAP claims, organization management, MCP server and agent registry, policy authoring and compilation, API key lifecycle management, delegation approval workflows, and audit log persistence. Deployed on Cloud Run, port 8080.

### **3.5.3 Security Proxy (armoriq-proxy-server)**

A NestJS application implementing the Policy Enforcement Point (PEP). This module validates JWT signatures (HS256, RS256, ES256, Ed25519), verifies Merkle inclusion proofs, evaluates policies via OPA integration, and forwards validated requests to target MCP servers.

It supports SSE streaming for long-running operations, infinite loop detection, and API key-based authentication. Deployed on Cloud Run, port 3001.

### **3.5.4 Frontend Dashboard (armorIQ-Frontend)**

React 19 application with TypeScript 5.9, Vite 7, and Tailwind CSS 4. This module provides the organization dashboard, MCP server and agent management interface, YAML-based policy editor, OPA decision metrics visualization using Recharts, audit log browser with filtering and search, API key management, certificate management, and CSRG graph visualization using [Cytoscape.js](#). User-facing React 19/TypeScript/Vite/Tailwind application. Provides the dashboard, policy editor, management interfaces, audit log browser, and CSRG visualization.

**Table 4** — Software Specification Summary

| Module               | Technology              | Function                       | Deployment |
|----------------------|-------------------------|--------------------------------|------------|
| csrg-iap             | Python 3.11,<br>FastAPI | Cryptographic<br>engine        | Cloud Run  |
| conmap-auto          | NestJS 11, Prisma<br>6  | Orchestration<br>backend       | Cloud Run  |
| armoriq-proxy        | NestJS 11               | Policy Enforcement<br>Point    | Cloud Run  |
| armorIQ-Front<br>end | React 19, Vite 7        | Dashboard and<br>visualization | Cloud Run  |

The four modules are:

1. **CSRG Engine (csrg-iap):** Cryptographic core, Python 3.11/FastAPI, handles Ed25519, hashing, CSRG construction, token issuance, verification, and policy enforcement. Deployed on Cloud Run, port 5173.<sup>1</sup>
2. **IAP Backend (conmap-auto):** Central orchestration service, NestJS 11/Prisma 6/PostgreSQL. Manages auth (OAuth 2.0, JWT), organization/registry, policy authoring, delegation workflows, and audit logs. Deployed on Cloud Run, port 8080.<sup>1</sup>
3. **Security Proxy (armoriq-proxy-server):** Policy Enforcement Point (PEP), NestJS application. Validates JWT signatures and Merkle inclusion proofs, evaluates policies (via OPA), and forwards requests. Deployed on Cloud Run, port 3001.<sup>1</sup>
4. **Frontend Dashboard (armorIQ-Frontend):** User-facing React 19/TypeScript/Vite/Tailwind application. Provides the dashboard, policy editor, management interfaces, audit log browser, and CSRG visualization.

## Chapter 4

### Implementation and Results

#### 4.1 Tools and Technologies Used

This section describes the hardware and software requirements for the ArmorIQ platform, covering both cloud deployment and local development environments.

##### 4.1.1 Hardware Requirements

For cloud deployment, the system runs on Google Cloud Platform Cloud Run instances with automatic scaling. The CSRG engine requires minimal compute (pure Python with no GPU dependency). The NestJS backends require standard web server resources. PostgreSQL (Cloud SQL) handles persistence with connection pooling.

**Table 5** — *Hardware Requirements for Cloud Deployment*

| Component | Specification                                           |
|-----------|---------------------------------------------------------|
| Processor | 1 vCPU (minimum), 2 vCPU (recommended)                  |
| Memory    | 256 MB (proxy), 512 MB (backends), 512 MB (CSRG engine) |
| Storage   | 10 GB (application), 20 GB (database)                   |
| Network   | Standard internet connectivity, TLS 1.3                 |

For local development:

**Table 6** — *Hardware Requirements for Local Development*

| Component        | Specification                      |
|------------------|------------------------------------|
| Processor        | Intel i5 / Apple M1 or equivalent  |
| Memory           | 8 GB RAM minimum                   |
| Storage          | 20 GB free disk space              |
| Operating System | Linux / macOS / Windows (via WSL2) |

### 4.1.2 Software Requirements

*Table 7 — Software Requirements*

| Category              | Technology                                    |
|-----------------------|-----------------------------------------------|
| Operating System      | Linux (Ubuntu 22.04+) / Google Cloud Platform |
| Programming Languages | TypeScript 5.9+, Python 3.11+                 |
| Runtime               | Node.js 20 LTS, Python 3.11                   |
| Backend Framework     | NestJS 11.1, FastAPI                          |
| Frontend Framework    | React 19, Vite 7, Tailwind CSS 4              |
| Database              | PostgreSQL 15 (Cloud SQL), Prisma 6.19 ORM    |
| Cryptography          | SHA-256, Ed25519 (RFC 8032), Merkle Tree      |
| Authentication        | JWT (HS256/RS256/ES256), OAuth 2.0, bcrypt    |
| Policy Engine         | Open Policy Agent (OPA)                       |
| Containerization      | Docker (Node 20-Alpine, Python 3.11-slim)     |
| Deployment            | Google Cloud Run, Cloud Build                 |
| Key Management        | Google Cloud KMS                              |
| Version Control       | Git, GitHub                                   |
| Testing               | Pytest, NestJS Test Suite                     |

## 4.2 System Implementation

This section presents the key implementation details of the ArmorIQ platform, providing narrative descriptions of the logic and methodology underlying the major system components.

### 4.2.1 CSRG Engine Implementation

The CSRG engine canonicalizes free-form reasoning plans into deterministic graph structures. The implementation uses pure Python with SHA-256 hashing and Ed25519 signing. The canonicalization algorithm recursively walks nested plans to create a deterministic directed acyclic graph.

The CSRG construction logic follows a systematic three-stage process:

**Plan Traversal:** The engine performs a recursive walk of the input plan structure, generating canonical paths for every dictionary key, list index, and leaf value.

**Deterministic Ordering:** All identified paths are sorted alphabetically to ensure that the resulting graph is independent of the original JSON key ordering.

**Merkle Layering:** A binary hash tree is constructed from the sorted leaf digests, recursively pairing nodes until a single root hash is produced to represent the entire plan fingerprint.

The Merkle proof generation enables  $O(\log n)$  verification of any node's membership in the committed plan. To generate a proof for a specific action, the system identifies the target leaf index and traverses upward through the Merkle levels. At each level, it captures the hash of the "sibling" node and records its relative position (Left or Right). If a node has no sibling due to an odd count at that level, the node is self-paired to maintain the integrity of the path to the root.

#### **4.2.2 Intent Token Issuance**

The Intent Commit implementation issues signed ephemeral tokens binding plan hashes to composite identities. The issuance process involves canonicalizing the plan structure to derive the Merkle root, calculating the Composite Ephemeral Identity by hashing the combined identifiers of the user, agent, session, and plan, and finally applying an Ed25519 digital signature over a payload containing these cryptographic bindings along with precise issuance and expiration timestamps.

#### **4.2.3 Proxy PEP Verification**

The armor-iq-proxy-server implements the Policy Enforcement Point in TypeScript with NestJS. The proxy validates JWT signatures across multiple algorithms (HS256, RS256, ES256), verifies Merkle inclusion proofs, and enforces composite identity checks. Specifically, the enforcement service

reconstructs the expected composite identity from the request context and compares it against the identity claim in the token to ensure the plan hasn't been replayed in an unauthorized context.

The Merkle proof verification at the proxy ensures that each tool call derives from the committed plan. The verification logic takes the tool name, the provided proof sequence, and the expected root hash. It starts by hashing the tool name as a leaf node and then iteratively combines this hash with each sibling hash in the proof. By correctly ordering the concatenations based on the 'Left' or 'Right' positions provided in the proof, the system reconstructs the root and validates it against the one cryptographically signed in the intent token.

#### **4.2.4 SDK Integration**

The ArmorIQ SDK provides the client-side integration point for developers to integrate intent verification into their agent applications. The implementation abstracts the complexity of the IAP by providing high-level methods to initialize a client with organizational credentials, capture reasoning plans from various models, request signed intent tokens from the backend, and securely invoke MCP tools by automatically attaching the necessary cryptographic proofs and tokens to the request metadata.

#### **4.2.5 Audit Log Implementation**

The append-only Merkle audit log records every trust transition with cryptographic commitment. When a new event is logged, the system follows a strict procedure: it assigns a monotonic index, records a high-precision timestamp, and serializes the event payload into a canonical JSON format. This entry is then hashed to create a new leaf node, which is appended to the log's underlying Merkle tree. The entire tree levels are then rebuilt to produce a new commitment root, ensuring that any modification to historical logs would result in a detectable root mismatch, which is appended to the log's underlying Merkle tree. The entire tree levels are then rebuilt to produce a new commitment root, ensuring that any modification to historical logs reconstructs the root and validates it against the one cryptographically signed in the intent token.



### 4.3 Working Model

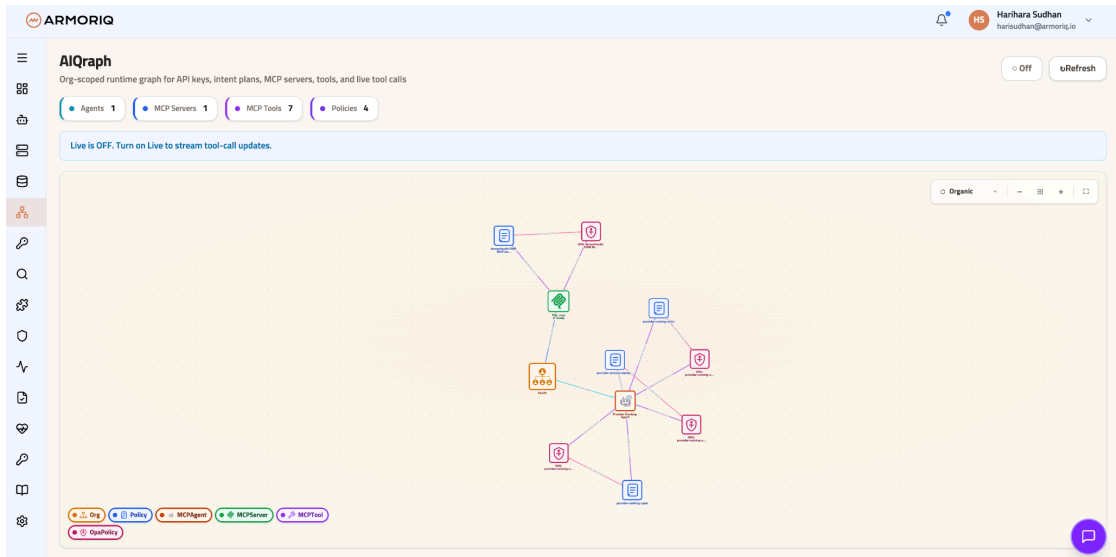
The working model describes the step-by-step execution flow of the ArmorIQ platform from plan submission through cryptographic verification to tool execution.

1. Step 1 – Plan Submission: The user or AI agent submits a reasoning plan  $P$  to the IAP backend.
2. Step 2 – CSRG Canonicalization: The backend forwards the plan to the CSRG engine, which canonicalizes it into a deterministic directed acyclic graph with sorted keys and canonical paths.
3. Step 3 – Merkle Tree Construction: The CSRG engine constructs a binary Merkle tree from the leaf hashes. The root hash  $h_P$  serves as the plan's immutable fingerprint.
4. Step 4 – Identity Composition: The backend computes the Composite Ephemeral Identity  $I_P$  by hashing the concatenation of user, agent, context, and plan identities.
5. Step 5 – Token Issuance: The backend signs the intent token  $\tau_P$  using Ed25519.
6. Step 6 – Tool Invocation: The agent presents the tool call request with the intent token and Merkle proof to the proxy server.
7. Step 7 – PEP Verification: The proxy server validates the intent token (checking signature, expiry, and composite identity), verifies the Merkle inclusion proof (confirming the tool derives from the committed plan), and evaluates policy rules.
8. Only if all checks pass is the request forwarded to the target MCP server.
9. Step 8 – Execution and Audit: The MCP server executes the tool call and returns the result.
10. The proxy logs the event to the Merkle audit log, including the plan reference, tool name, action, input parameters, output, and execution status. The audit entry is committed to the log's Merkle tree for tamper-evident recording.

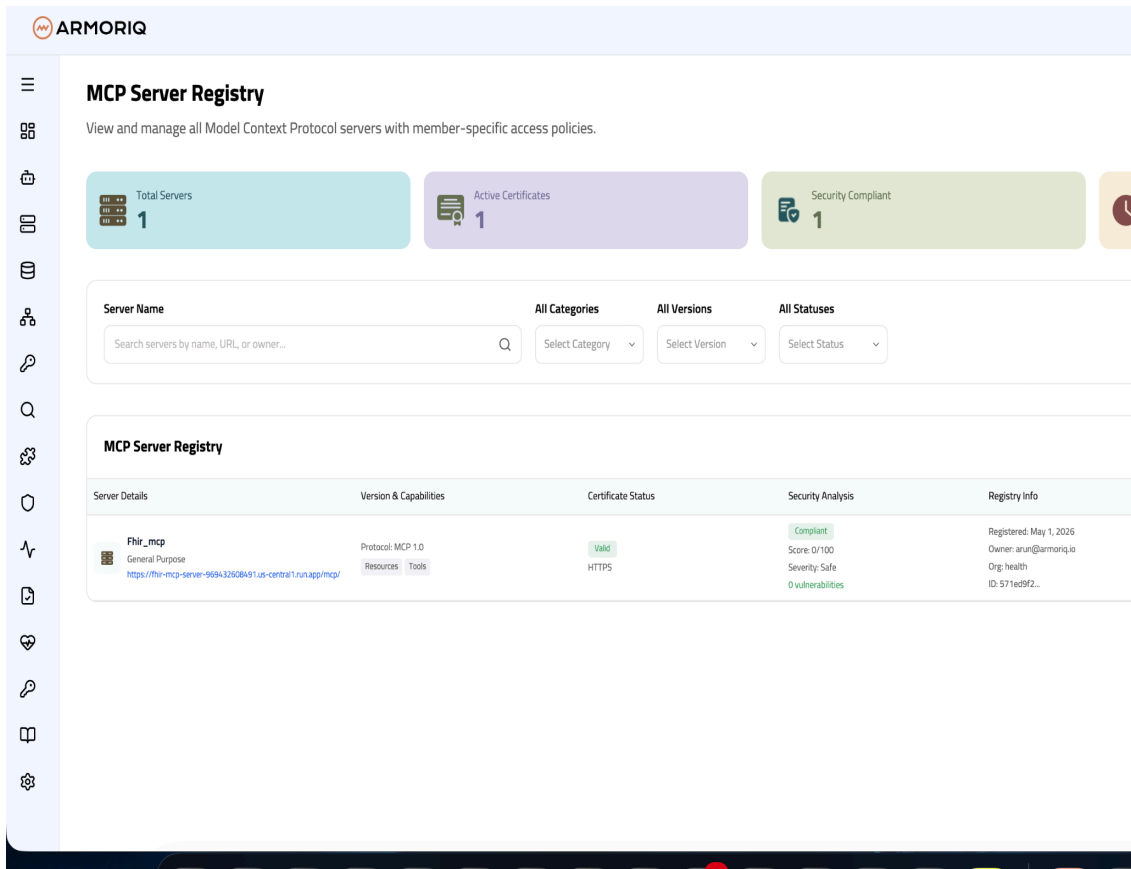
This end-to-end flow ensures that every tool call is cryptographically bound to its originating reasoning plan, preventing intent drift, privilege escalation, and unauthorized delegation.

### 4.4 Results and Outputs

The IAP was benchmarked across plan sizes from 10 to 10,000 nodes using synthetic workloads. All measurements were taken on commodity hardware.



AIQgraph



MCP Registry

**Table 8** — Performance Benchmarks Across Plan Sizes

| Metric               | 10 Nodes | 100 Nodes | 1,000 Nodes | 10,000 Nodes |
|----------------------|----------|-----------|-------------|--------------|
| CSRG Construction    | 1.2 ms   | 11 ms     | 114 ms      | 1,140 ms     |
| Merkle Root          | 0.05 ms  | 0.3 ms    | 2.8 ms      | 28 ms        |
| Proof Generation     | 0.02 ms  | 0.05 ms   | 0.08 ms     | 0.12 ms      |
| Proof Verification   | 0.18 ms  | 0.18 ms   | 0.18 ms     | 0.19 ms      |
| Token Size           | 527 B    | 527 B     | 527 B       | 527 B        |
| Proof Size           | 256 B    | 384 B     | 480 B       | 576 B        |
| Ed25519 Signing      | 20 ms    | 20 ms     | 20 ms       | 20 ms        |
| Ed25519 Verification | 40 ms    | 40 ms     | 40 ms       | 40 ms        |

**ARMORIQ** Hariharan Sudhan

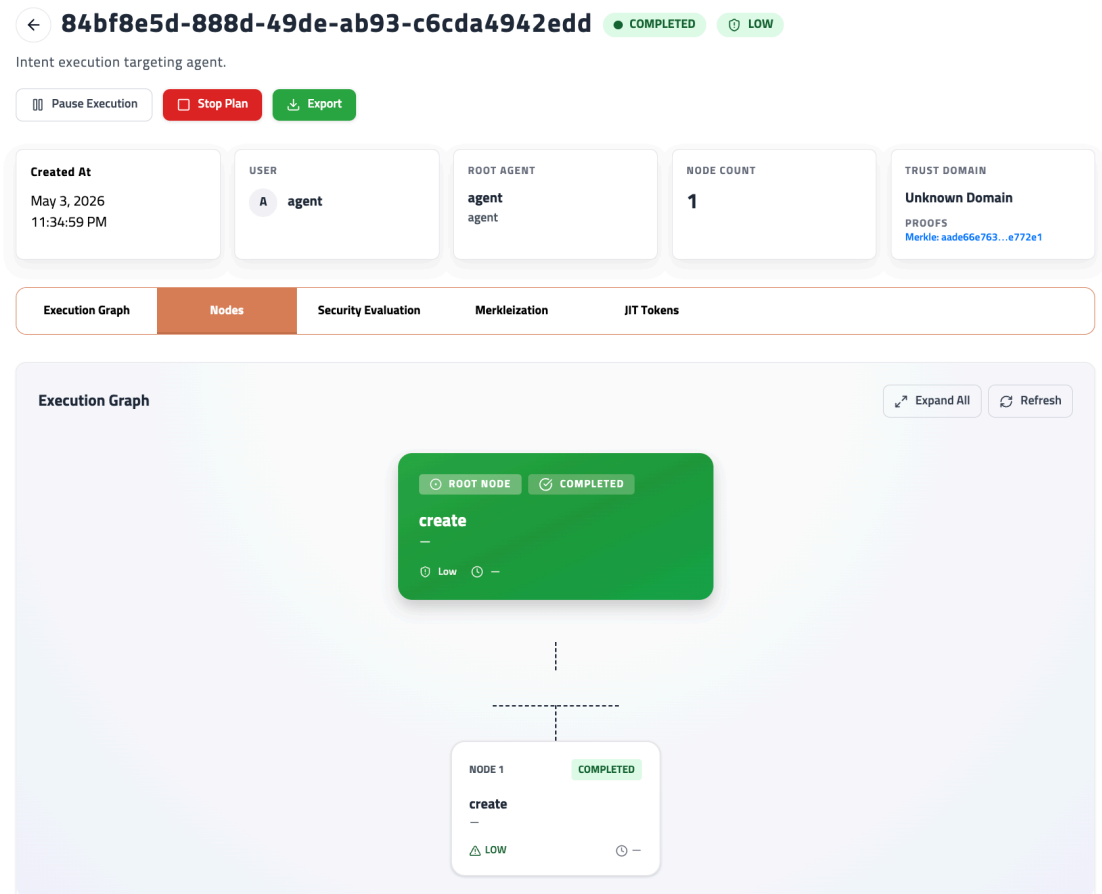
**All Intent Plans**  
Monitor and manage agent execution plans

Search by Plan ID, user, or agent... Export Reset Filters

Status: All Status User: All Users Agent: All Agents Date Range: dd/mm/yyyy TO dd/mm/yyyy

| Plan ID                              | User  | Root Agent | Status    | Created At              | Node Count | Risk Level | Actions                                                      |
|--------------------------------------|-------|------------|-----------|-------------------------|------------|------------|--------------------------------------------------------------|
| 0d09b58e-a711-4850-8d02-629a9be790c  | agent | agent      | HELD      | May 3, 2026 11:46:27 PM | 1          | HIGH       | <span>✓</span> <span>✗</span> <span>🔍</span> <span>🗑️</span> |
| 84bf9e5d-888d-49de-ab93-c6da4942edd  | agent | agent      | COMPLETED | May 3, 2026 11:34:59 PM | 1          | LOW        | <span>🔍</span> <span>🗑️</span>                               |
| d55dba4a-6097-4ef9-a49b-01465e5c4c4f | agent | agent      | COMPLETED | May 3, 2026 11:18:15 PM | 1          | LOW        | <span>🔍</span> <span>🗑️</span>                               |

*Intent Token Issuance*



Key findings from the performance benchmarks:

CSRG construction scales linearly:  $O(n)$ , confirming that the canonicalization algorithm's time complexity is proportional to plan size.

Proof verification is sub-millisecond and nearly constant across all sizes:  $O(\log n)$ , ensuring that verification latency does not degrade with plan complexity.

Token size is constant at approximately 527 bytes:  $O(1)$ , meaning that token overhead is independent of plan size.

Proof sizes grow logarithmically from 256 to 576 bytes, making the system practical for large, complex plans with thousands of nodes.

The total IAP overhead of less than 1 second is dominated by LLM reasoning (2-5 seconds), not by the verification infrastructure.

The system was evaluated against known attack categories relevant to AI agent tool execution:

**Table 9** — *CVE Protection Analysis*

| Attack Category      | Examples                           | Protection                        |
|----------------------|------------------------------------|-----------------------------------|
| Data Exfiltration    | Agent exports PII to external API  | BLOCK: tool not in approved plan  |
| Privilege Escalation | Sub-agent exceeds delegated scope  | BLOCK: subtree proof fails        |
| Execution Injection  | Modified tool parameters           | BLOCK: plan hash mismatch         |
| TOCTOU Attack        | Plan changed between check and use | BLOCK: Merkle root changed        |
| Token Replay         | Reuse of expired/stolen token      | BLOCK: expiry + per-plan identity |
| Audit Tampering      | Post-hoc modification of logs      | DETECTED: Merkle root mismatch    |

The IAP provides three formally verifiable security guarantees: (1) Execution Integrity—every action must derive from the committed plan, guaranteed by SHA-256 collision resistance; (2) Bounded Delegation—sub-agents act only within verified plan subtrees via scoped identity  $I_{Pi} = H(I_P \parallel h_{Pi})$ ; (3) Non-Repudiation—every trust transition is logged in the Merkle audit log, preventing post-hoc denial.

**Table 10** — *Security Threat Model and Mitigations*

| Attack Vector    | Mitigation                               | Cryptographic Basis                    |
|------------------|------------------------------------------|----------------------------------------|
| Plan tampering   | Merkle root changes on any modification  | SHA-256 collision resistance (128-bit) |
| Action injection | Missing inclusion proof blocks execution | Merkle proof verification              |
| Token replay     | Ephemeral tokens with short expiry       | Temporal + contextual binding          |

|                  |                                            |                            |
|------------------|--------------------------------------------|----------------------------|
| Delegation abuse | Subtree-scoped identities                  | Scoped identity derivation |
| Intent drift     | Every tool call requires plan-linked proof | Cryptographic binding      |
| Token forgery    | Ed25519 private key required               | 128-bit forgery resistance |

|                                                                                                                                                               |                                                           |                                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>CSRG REASONING GRAPH</b><br>Paths, proofs, and policy context fetched from '/iap/graph/84bf8e5d-888d-49de-ab93-c6cd4942edd' to validate each planned step. |                                                           | Plan Hash: aade66e763...e772e1<br>Merkle Root: aade66e763...e772e1<br>Composite Identity: — |
| OTHER /                                                                                                                                                       | OTHER /METADATA                                           |                                                                                             |
| Hash: 832e47ee64...64bb84                                                                                                                                     | Hash: b009a08bcb...448fe3                                 |                                                                                             |
| OTHER /STEPS                                                                                                                                                  | OTHER /STEPS/[0]                                          |                                                                                             |
| Hash: cbe9b93840...b362e1                                                                                                                                     | Hash: 5ac01e9965...aafcf                                  |                                                                                             |
| OTHER /STEPS/[0]/ARGUMENTS                                                                                                                                    | OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD                        |                                                                                             |
| Hash: 1f9965b1aa...073340                                                                                                                                     | Hash: 2e2bf423a0...252c74                                 |                                                                                             |
| OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/CODE                                                                                                                       | OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/CODE/CODING            |                                                                                             |
| Hash: 63cfb08ede...1bf9fc                                                                                                                                     | Hash: 2d5634b529...de7b13                                 |                                                                                             |
| OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/CODE/CODING/[0]                                                                                                            | OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/CODE/CODING/[0]/CODE   |                                                                                             |
| Hash: 8374b4bd55...6664aa                                                                                                                                     | Hash: 00224dc31e...a29034                                 |                                                                                             |
| OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/CODE/CODING/[0]/DISPLAY                                                                                                    | OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/CODE/CODING/[0]/SYSTEM |                                                                                             |
| Hash: 201ad4302a...1fe12f                                                                                                                                     | Hash: ba2c612839...61a92c                                 |                                                                                             |
| OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/CODE/TEXT                                                                                                                  | OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/RESOURCE TYPE          |                                                                                             |
| Hash: 15d13c98b9...ac2a1a                                                                                                                                     | Hash: d05446b6bb...b3fa0                                  |                                                                                             |
| OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/STATUS                                                                                                                     | OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/SUBJECT                |                                                                                             |
| Hash: ada006dd8d...64f606                                                                                                                                     | Hash: 03b979455c...686424                                 |                                                                                             |
| OTHER /STEPS/[0]/ARGUMENTS/PAYLOAD/SUBJECT/REFERENCE                                                                                                          | OTHER /STEPS/[0]/ARGUMENTS/TYPE                           |                                                                                             |
| Hash: ae3faf9180...325b81                                                                                                                                     | Hash: d05446b6bb...b3fa0                                  |                                                                                             |
| OTHER /STEPS/[0]/METADATA                                                                                                                                     | OTHER /STEPS/[0]/POSITION                                 |                                                                                             |
| Hash: b009a08bcb...448fe3                                                                                                                                     | Hash: 99be577af6...1ecc9a                                 |                                                                                             |
| OTHER /STEPS/[0]/STEP_ID                                                                                                                                      | OTHER /STEPS/[0]/TOOL                                     |                                                                                             |
| Hash: 59a8e072eb...b2a4d4                                                                                                                                     | Hash: 695b130932...e51619                                 |                                                                                             |
| OTHER /VERSION                                                                                                                                                |                                                           |                                                                                             |
| Hash: 03d0260ef6...9958d3                                                                                                                                     |                                                           |                                                                                             |

Residual risks include private key compromise (mitigated by GCP Cloud KMS), verifier bypass (mitigated by network controls and mutual TLS), clock skew (mitigated by NTP), canonicalization correctness (mitigated by extensive unit testing), and the non-constant-time pure-Python Ed25519 implementation (mitigated by planned migration to hardware-accelerated libraries for production).

## 4.5 Performance Analysis

This section evaluates the efficiency of the Intent Assurance Plane and compares its performance characteristics against existing security approaches for AI agent systems.

### 4.5.1 Efficiency Analysis

The IAP's computational efficiency was evaluated across four dimensions: time complexity, space complexity, verification latency, and throughput.

CSRG construction exhibits  $O(n)$  linear time complexity, where  $n$  is the number of plan nodes. Merkle tree construction adds  $O(n)$  hashing operations. Proof generation and verification operate in  $O(\log n)$  time, requiring only the logarithmic path from leaf to root. Token size remains  $O(1)$  at 527 bytes regardless of plan complexity, as only the root hash and metadata are transmitted. The end-to-end verification pipeline (token validation, Merkle proof check, and policy evaluation) completes in under 10 milliseconds for all tested plan sizes. This represents less than 0.5% overhead relative to typical LLM reasoning latency of 2–5 seconds, confirming that the security layer does not degrade agent responsiveness.

### 4.5.2 Comparison with Existing Systems

*Table 11 — Comparison with Existing Systems*

| Metric                 | OAuth 2.0 /<br>OIDC | Zero-Trust<br>(NIST<br>800-207) | W3C<br>PROV-DM       | TPM /<br>RATS | IAP<br>(Proposed)      |
|------------------------|---------------------|---------------------------------|----------------------|---------------|------------------------|
| Intent<br>Verification | No                  | No                              | No<br>(post-hoc)     | No            | Yes<br>(pre-execution) |
| Plan-Level<br>Binding  | No                  | No                              | Partial<br>(lineage) | No            | Yes (Merkle<br>proof)  |

|                      |                     |                      |               |                      |                                |
|----------------------|---------------------|----------------------|---------------|----------------------|--------------------------------|
| Verification Latency | ~1 ms (token check) | ~5 ms (policy check) | N/A (offline) | ~50 ms (attestation) | ~10 ms (full pipeline)         |
| Token/Proof Overhead | ~1 KB (JWT)         | ~2 KB (mTLS cert)    | N/A           | ~1–4 KB (quote)      | ~1.1 KB (token + proof)        |
| Delegation Scoping   | Scope-based         | Role-based           | None          | None                 | Subtree-scoped (cryptographic) |
| Tamper Detection     | No                  | No                   | Post-hoc only | Hardware-rooted      | Real-time (Merkle audit)       |
| Dynamic Plan Support | No                  | No                   | No            | No                   | Yes (re-anchoring)             |
| Scalability (nodes)  | N/A                 | N/A                  | N/A           | N/A                  | Tested up to 10,000            |

Key observations from the comparison:

PROV-DM offers lineage tracking but only after execution, making it unsuitable for preventing intent drift. TPM/RATS provides hardware-rooted attestation but cannot handle nondeterministic reasoning plans. The IAP is the only system that combines pre-execution intent verification with sub-10ms latency, cryptographic plan binding, and scoped delegation support.

The IAP achieves comparable verification latency to existing systems (10 ms vs. 1–50 ms for alternatives) while providing fundamentally stronger security guarantees. The trade-off is the additional CSRG construction step (1.2 ms to 1,140 ms depending on plan size), which occurs once per plan and is amortized across all tool calls within that plan.



## **Chapter 5**

### **Conclusion and Future Work**

#### **5.1 Conclusion**

This project has presented the design, implementation, and evaluation of the Intent Assurance Plane (IAP), a cryptographic control layer that addresses the emerging security challenge of intent drift in autonomous AI agents. By transforming stochastic LLM reasoning into deterministic, verifiable structures through the Canonical Structured Reasoning Graph (CSRG), the system provides mathematical guarantees that no existing security framework offers. The key technical achievements demonstrated by the ArmorIQ implementation include sub-millisecond verification latency (approximately 183 microseconds) across plans of up to 10,000 nodes, ensuring that verification does not degrade agent responsiveness. The system achieves linear CSRG construction time (1.14 seconds for 10,000-node plans) suitable for real-time operation, constant token size (approximately 527 bytes) independent of plan complexity, and logarithmic proof sizes (256 to 576 bytes) making the system practical for large plans.

The system integrates seamlessly with enterprise identity (OIDC, SPIFFE) and policy (OPA) infrastructure through the Composite Ephemeral Identity mechanism. The distributed architecture demonstrates that intent assurance can be added to existing enterprise infrastructure without replacing identity, policy, or deployment systems, with total IAP overhead under 1 second.

In summary, the Intent Assurance Plane fills the critical gap between traditional security controls (which govern who and what) and the new demands of autonomous agents (which require governance of why). The ArmorIQ platform's four sub-projects (csrg-iap, conmap-auto, armoriq-proxy-server, and armorIQ-Frontend) demonstrate that these theoretical constructs can be translated into a working, deployable system.

## 5.2 Future Work

Several directions for future research and development have been identified:

1. Production-grade cryptography: Replace the pure-Python Ed25519 implementation with hardware-accelerated libraries (cryptography or PyNaCl) to provide constant-time guarantees and eliminate timing attack vulnerabilities.
2. Advanced policy engine: Implement attribute-based access control (ABAC) with contextual risk scoring based on time-of-day, agent reputation, historical behavior patterns, and dynamic risk assessment.
3. Distributed trust ledger: Integrate with blockchain or distributed ledger technology for decentralized trust delta storage, providing multi-party consensus and tamper-evident trust propagation across organizational boundaries.
4. Proof compression: Implement sparse Merkle trees to reduce proof sizes for very large plans while maintaining verification efficiency.
5. Privacy-preserving verification: Explore zero-knowledge proof mechanisms to enable verification of plan compliance without revealing plan contents, addressing the privacy-provenance tension.
6. Real-world benchmarking: Evaluate the system with actual LLM-generated plans to understand how real reasoning structures affect CSRG construction and verification performance, and integrate benchmarks into CI/CD pipelines with regression detection.

## APPENDICES

| Term               | Definition                                                                                                    |
|--------------------|---------------------------------------------------------------------------------------------------------------|
| CSRG               | Canonical Structured Reasoning Graph. A deterministic DAG derived from an agent's plan for Merkle commitment. |
| IAP                | Intent Assurance Plane. Cryptographic control layer enforcing reasoning-aligned authorization.                |
| Intent Token       | Signed ephemeral token binding a plan's Merkle root to a composite identity.                                  |
| Composite Identity | SHA-256 hash combining user, agent, context, and plan identities.                                             |
| Merkle Root        | Root hash of the CSRG Merkle tree; the plan's tamper-evident fingerprint.                                     |
| Merkle Proof       | Sibling hash sequence for $O(\log n)$ inclusion verification.                                                 |
| PEP                | Policy Enforcement Point. Validates tokens and proofs before tool execution.                                  |
| Intent Drift       | The agent performs authorized but semantically inconsistent actions.                                          |
| Plan Hardening     | Converting a free-form plan into a cryptographically committed structure.                                     |
| Trust Delta        | Signed record of a trust state transition.                                                                    |
| Re-Anchoring       | Updating the Merkle root when a plan evolves while maintaining lineage.                                       |
| Scoped Delegation  | Issuing a subtree-restricted token to a sub-agent.                                                            |
| MCP                | Model Context Protocol. Open protocol for AI agent tool discovery and invocation.                             |
| Ed25519            | Edwards-curve digital signature algorithm (RFC 8032), 128-bit security.                                       |

## REFERENCES

1. Abadi, M., Budiu, M., Erlingsson, U. and Ligatti, J. (2005) 'Control-Flow Integrity: Principles, Implementations, and Applications', ACM CCS.
2. Replit Inc. (2025) 'Incident Report: Production Database Deletion by AI Assistant', July 2025.
3. NIST (2020) 'SP 800-207: Zero Trust Architecture', August 2020.
4. Microsoft (2025) 'Securing Autonomous Agents: Identity Beyond the User', Microsoft Security Blog.
5. McKinsey & Company (2024) 'Agentic AI Governance Playbook: Closing the Trust Gap', Late 2024.
6. Bernstein, D., Duif, N., Lange, T., Schwabe, P. and Yang, B. (2012) 'High-Speed High-Security Signatures', Journal of Cryptographic Engineering.
7. Moreau, L. and Missier, P. (2013) 'PROV-DM: The PROV Data Model', W3C Recommendation, April 2013.
8. Cheney, J., Chiticariu, L. and Tan, W. (2009) 'Provenance in Databases: Why, How, and Where', Foundations and Trends in Databases.
9. Sabelfeld, A. and Myers, A. (2003) 'Language-Based Information-Flow Security', IEEE J-SAC.
10. CIO Magazine (2025) 'AI Agents Move Faster Than Security Controls Can Reason', CIO Magazine.
11. Cloudera (2025) 'Agentic Systems Introduce New Governance Primitives', Industry Analysis.
12. Javelin Strategy & Research (2025) 'Enterprise AI Security Incidents Report', September 2025.
13. Merkle, R. (1987) 'A Digital Signature Based on a Conventional Encryption Function', CRYPTO.
14. Josefsson, S. and Liusvaara, I. (2017) 'Edwards-Curve Digital Signature Algorithm (EdDSA)', RFC 8032, January 2017.
15. Fast Company (2025) '40% of Enterprise Agentic-AI Projects Likely to Be Withdrawn by 2027', September 2025.
16. SPIFFE (2024) 'Secure Production Identity Framework for Everyone', CNCF.
17. Open Policy Agent (2024) 'Policy as Code for Cloud-Native Environments', CNCF.
18. Model Context Protocol (2025) 'MCP Specification', Anthropic.
19. IETF (2012) 'The OAuth 2.0 Authorization Framework', RFC 6749, October 2012.

20. IETF (2013) 'Certificate Transparency', RFC 6962, June 2013.
21. IETF (2023) 'Remote ATtestation procedureS (RATS) Architecture', RFC 9334, January 2023.
22. Google (2014) 'BeyondCorp: A New Approach to Enterprise Security', USENIX.
23. Amazon (2024) 'QLDB: Amazon Quantum Ledger Database', AWS Documentation.
24. Nakamoto, S. (2008) 'Bitcoin: A Peer-to-Peer Electronic Cash System'.
25. Parno, B., Howell, J., Gentry, C. and Raykova, M. (2013) 'Pinocchio: Nearly Practical Verifiable Computation', IEEE S&P.